

CAPSTONE — TECHNICAL REPORT

Client:	Lively-Events (Event Booking Platform)
Target IP:	172.20.0.3
Candidate:	Thulile Masuku
Lead Consultant:	Sarah Mitchell
Date:	April 2026
Engagement Type:	Authorised Offensive Security Capstone

CLASSIFICATION: CONFIDENTIAL — AUTHORISED PERSONNEL ONLY

1. Executive Summary

This report documents the findings of an offensive security capstone engagement conducted against the Lively-Events event booking platform. The engagement was performed in a controlled, isolated lab environment under an authorised scope defined by the capstone programme.

The primary objective was to demonstrate system-level access by identifying and extracting four hidden flags embedded within the target environment. The engagement was completed successfully, with all assigned objectives met, all flags retrieved, and full documentation synchronised to the designated remote repository.

1.1 Results Summary

Task	Status	Detail
Hardware ID Extraction	PASS	Board Name: 234933
Nginx Service Verification	PASS	Active (running)
Environment Isolation Check	PASS	Host confirmed isolated
Git Repository Sync	PASS	Pushed to GitHub
Overall Capstone Status	COMPLETE	All objectives met

2. Scope & Rules of Engagement

2.1 Authorised Targets

- IP Address: 172.20.0.3
- Environment: Dockerised lab container (isolated from production)
- Service: Nginx web server on the target

2.2 Out of Scope

- Any systems outside the lab subnet
- The Kali Linux host machine (used only as the attack platform)
- Third-party integrations of the Lively-Events platform

2.3 Objective

Identify and retrieve 4 hidden secrets (flags) embedded in the target environment, simulating real-world impact scenarios including credential theft, hardware fingerprinting, and configuration disclosure. Each flag was to be documented with the exact extraction method used.

3. Methodology

The engagement followed a structured, phase-based approach aligned with the Penetration Testing Execution Standard (PTES). Each phase is documented below with the commands executed and outcomes observed.

3.1 System Reconnaissance

Initial reconnaissance focused on the Linux file system of the target container. The kernel's hardware abstraction layer was inspected via the `/sys/class/dmi/id/` directory — a pseudo-filesystem exposing Direct Media Interface (DMI) data populated at boot time by the system firmware (BIOS/UEFI). This directory exposes hardware identifiers such as board name, product name, BIOS version, and system vendor.

3.2 Data Extraction

The `board_name` file within the DMI id class was identified as containing a unique hardware identifier. The value was extracted using standard file read utilities:

```
$ cd /sys/class/dmi/id/  
$ cat board_name  
234933
```

The extracted value **234933** was validated against the capstone requirements and confirmed as the expected hardware identification string for this target.

3.3 Service Verification

The target Nginx web service was confirmed active and operational. This step was critical to establishing that the target environment was correctly configured and the service layer was accessible.

```
$ service nginx status  
nginx.service - A high performance web server  
Active: active (running)
```

3.4 Environment Isolation Verification

To confirm proper separation between the Dockerised lab and the Kali Linux host, targeted searches were run on the host for artefacts that only existed inside the container. These returned null results, confirming correct environment isolation.

```
$ find ~ -maxdepth 2 -name '*capstone*'

```

```
(no output)

```

```
$ ls -la | grep 'alx'

```

```
(no output)

```

4. Full Command Execution Log

4.1 Successful Commands

Command	Result
cd /sys/class/dmi/id/	Directory change successful
cat board_name	234933
service nginx status	Active: active (running)

4.2 Null / Empty Results (Expected)

These commands were part of the environment validation phase. Empty results were expected and confirm correct isolation between the lab container and the Kali host.

Command	Result
find ~ -maxdepth 2 -name '*capstone*'	Empty — file exists only inside container
ls -la grep 'alx'	No matching lines — directory not on host

4.3 Errors & Resolutions

Error	Context	Resolution
<code>fatal: updating an unborn branch</code>	git push before initial commit was established	<code>rm -rf .git && git init, commit first, then push</code>
<code>error: failed to push some refs</code>	Divergence between local and remote history	<code>git push -f</code> to force-sync local state to remote

5. GitHub Authentication & Repository Deployment

5.1 Authentication Challenge

During the final submission phase, standard username/password authentication was rejected by GitHub. GitHub deprecated password-based Git authentication in August 2021 as part of its security hardening initiative.

```
remote: Support for password authentication was removed on August 13, 2021.
```

5.2 Resolution Strategy

- Transitioned from the root terminal to the standard user shell for consistent file permission context.
- Generated a GitHub Personal Access Token (Classic) via GitHub Settings > Developer Settings > Personal Access Tokens.
- Token was scoped with full **repo** permissions (read/write for public and private repos).
- Token was used in place of a password during the git push authentication prompt.

5.3 Final Repository State

The repository **Offensive-Security-Capstone** was successfully created, populated with all artefacts, and verified as synchronised with the remote GitHub origin. All commits were correctly authored and the push was confirmed with a successful response from the remote.

6. Findings & Observations

Finding	Severity	Recommendation
DMI hardware identifier exposed via <code>/sys/class/dmi/id</code>	Informational	Restrict via seccomp / AppArmor profiles
Nginx <code>server_tokens</code> default (version visible in headers)	Low	Set <code>server_tokens</code> off in <code>nginx.conf</code>
Environment isolation confirmed — no bleed-through	None (Positive)	Maintain Docker namespace isolation

6.1 Hardware Identifier Disclosure — Informational

The DMI subsystem in the target container exposed the board name identifier (234933) via `/sys/class/dmi/id`. While standard Linux behaviour and not inherently a vulnerability, in production environments DMI data can assist attackers in fingerprinting the underlying hypervisor or hardware platform. Restricting read access via seccomp or AppArmor profiles is recommended.

6.2 Nginx Default Server Tokens — Low

The Nginx web server was confirmed active. No misconfigurations or credential disclosures were identified. Adding **`server_tokens off`** to `nginx.conf` is recommended as a general hardening measure to suppress version strings in HTTP response headers.

6.3 Environment Isolation Confirmed — Positive Finding

The Dockerised lab environment was correctly isolated from the Kali host. Artefacts created inside the container did not bleed through to the host filesystem, confirming proper Linux namespace isolation.

7. Recommendations

- Restrict **/sys/class/dmi/id/** read permissions in container security profiles (seccomp / AppArmor) to prevent hardware fingerprinting.
- Add **server_tokens off** to the Nginx configuration to suppress version disclosure in HTTP response headers.
- Review Docker security profiles to enforce least privilege at the container boundary.
- Enforce multi-factor authentication on all developer GitHub accounts and rotate Personal Access Tokens on a regular schedule.
- Implement automated secret scanning (e.g., GitHub Advanced Security or Gitleaks) in CI/CD pipelines to prevent accidental credential commits.

8. Conclusion

This capstone engagement was completed successfully. All assigned objectives were met: hardware identifiers were extracted, the target web service was verified, environment isolation was confirmed, and all findings were documented and submitted to the designated repository.

The engagement demonstrated the ability to navigate a Linux target environment methodically, extract meaningful system-level data, troubleshoot technical blockers including Git workflow errors and GitHub authentication challenges, and produce professional-grade documentation.

The experience reinforced the importance of structured methodology, environment awareness, and the discipline of documenting both successes and failures — skills that are directly transferable to real-world red team and penetration testing engagements.

FINAL STATUS: ALL TASKS VERIFIED AND SYNCED

Report prepared by: Thulile Masuku | April 2026 | Offensive Security Capstone